VOL. 29

Cybersecurity

Defensive security from first principles to incident response

Source	tryhackme.com (gated; OWASP/defensive-primary fallback)
Generated	2026-10-10
Level	Beginner
Estimated study time	25 hours
Language	English
Volume	Vol. 29 - Cybersecurity

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 3

Chapter map . . . 3

Chapter 1 - Security thinking: assets, threats, vulnerabilities and risk . . . 4

1.1 Summary . . . 4

1.2 Concepts . . . 5

The four words that organise everything . . . 5

The CIA triad . . . 5

Defence in depth . . . 5

Types of control . . . 6

Risk treatment . . . 6

1.3 Examples . . . 6

1.4 Common mistakes and best practices . . . 8

1.5 Practice exercises . . . 8

Solutions . . . 8

1.6 Mini test . . . 9

Answer key and explanations . . . 9

Chapter 2 - Cryptography basics: hashing, encoding and randomness . . . 9

2.1 Summary . . . 10

2.2 Concepts . . . 10

What a hash function is . . . 10

Which algorithm for which job . . . 10

Hashing, encoding and encryption compared . . . 11

Salts, peppers and stretching . . . 11

Randomness . . . 11

2.3 Examples . . . 11

2.4 Common mistakes and best practices . . . 13

2.5 Practice exercises . . . 14

Solutions . . . 14

2.6 Mini test . . . 14

Answer key and explanations . . . 15

How to use this volume

Prerequisites. Basic computer literacy: you can install software, edit text files and move around a file system. A little networking (IP addresses, ports, DNS) and a little command-line experience help, but every idea is explained from the ground up. No security background is assumed.

Estimated study time. About 25 hours: roughly two hours per chapter, plus the time you spend on the exercises and on the small scripts. Chapters 1 to 6 build the core mental model; chapters 7 to 12 show how defenders work day to day.

How the chapters work. Every chapter opens with a summary, explains the concepts, shows numbered examples with the output they produce, lists common mistakes with the best practice for each one, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. The Python blocks tagged `py` are small teaching scripts. Save one as `script.py` and run `python script.py`. Blocks tagged `text` show the output you should see. Blocks tagged `powershell` or `bash` are reference commands for tools that are not part of this volume; they are labelled as not executed, because they need a real system to run against.

shell

```
python hashing_demo.py
```

output

```
message : correct horse battery staple
sha256  : c4bbcb1fbec99d65bf59d85c8cb62ee2db963f0fe106f483d9afa73bd4e39a8a
length  : 64 hex characters
```

Python 3 and the standard library are enough for every example; nothing needs to be installed.

A defensive promise. This volume teaches the defender point of view. It explains how attacks work in concept so that defences make sense, but it never provides step-by-step instructions for breaking into systems. The scripts run only against data they create themselves, or against the loopback address `127.0.0.1`, which always means your own machine.

About the source. The syllabus follows the TryHackMe learning paths named in the plan for this volume, starting with the free Pre Security and Cyber Security 101 paths. The path pages require an account, so the technical content here is written from well-known open references: the OWASP Top 10 and Cheat Sheet Series, MITRE ATT&CK, NIST publications and the CVSS specification. Nothing is copied from any of them.

Chapter map

#	Chapter	You will learn
1	Security thinking	the CIA triad, assets, threats, vulnerabilities, risk scoring
2	Cryptography basics	hashing, salting, encoding, randomness and key stretching

#	Chapter	You will learn
3	Cryptography basics	symmetric and asymmetric keys, certificates and TLS
4	Authentication	passwords, hashing choices, multi-factor authentication, sessions
5	Web attacks and defences	the OWASP Top 10 categories and the controls that stop them
6	Network security	ports, firewalls, segmentation, secure remote access
7	Malware	viruses, trojans, ransomware, spyware and the defences that work
8	Social engineering	phishing, pretexting, baiting and how people are defended
9	Vulnerability management	scanning, CVSS scoring, patch and exception workflows
10	Logging and the SOC	what to log, detection rules, alert triage
11	Hardening and response	baselines, incident response steps and recovery
12	Secure development	shift left, secure SDLC, NIST SSDF practices
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Security thinking: assets, threats, vulnerabilities and risk

1.1 Summary

- Security is the practice of protecting what matters from things that can go wrong, using a sensible budget of time, money and patience.
- The CIA triad names the three properties defenders try to preserve: confidentiality, integrity and availability.
- A threat is anything that could cause harm, a vulnerability is a weakness that lets the threat through, and a control reduces the chance or the damage.
- Risk is the combination of how likely harm is and how bad it would be, and it turns a long list of problems into an ordered plan.
- Defence in depth layers controls so that one broken control does not expose the whole system.
- Risk is handled in four ways only: avoid it, reduce it, transfer it, or accept it on purpose with a written reason.
- Security work never ends, because new systems, new features and new attackers keep changing the picture.

1.2 Concepts

The four words that organise everything

Every security conversation uses the same four words. Getting them straight makes the rest of the field much easier to read.

Word	Plain meaning	Example
Asset	something of value worth protecting	customer records, backup tapes, the build server
Threat	a person or event that could cause harm	a criminal gang, a careless employee, a storm
Vulnerability	a weakness that lets a threat in	an unpatched service, a shared password, a missing control
Risk	the chance of harm meeting its cost	a 40 percent chance of losing the customer database

A threat without a vulnerability is harmless; a vulnerability without a threat is boring. Risk is what you actually manage.

The CIA triad

Confidentiality, integrity and availability are the three classic goals. Every control in this volume protects at least one of them.

Goal	Question it answers	Everyday example
Confidentiality	can the wrong person read this?	only payroll staff can open the salary file
Integrity	can the data be changed without permission?	a checksum shows the download was not modified
Availability	can the right person use it when needed?	two web servers so one failure does not stop the shop

The three pull against each other. Locking every file in a safe improves confidentiality but destroys availability. Choosing between them is a business decision, not a technical one.

Defence in depth

One control is never enough. Layers are chosen so that an attacker who beats one layer still faces the next.

output

```
layer 7   policy and training     -> people know what to report
layer 6   monitoring and response -> the alarm rings while it happens
layer 5   application checks      -> input is validated before use
layer 4   operating system        -> patched, minimal, hardened
layer 3   network                 -> firewalls and segmentation
layer 2   physical                -> locked doors and badge readers
layer 1   data                   -> encrypted and backed up
```

Each layer assumes the one outside it might fail. That assumption is the whole idea.

Types of control

Type	Job	Example
Preventive	stops the event	firewall rule, least-privilege permission
Detective	sees that something happened	log alert, integrity check
Corrective	repairs after the fact	restore from backup, patch
Deterrent	discourages the attempt	warning banner, legal notice
Compensating	covers where the ideal control is missing	extra logging when a patch is impossible

Risk treatment

Treatment	Meaning	Example
Avoid	stop the activity that creates the risk	retire an old service nobody uses
Reduce	add controls to lower likelihood or impact	require multi-factor authentication
Transfer	move the cost to someone else	cyber insurance, a supplier clause
Accept	live with it and write down why	a low-value internal tool stays as is

Acceptance is a decision, not an oversight. It is written down with a name and a review date, otherwise it is simply neglect.

1.3 Examples

Example 1: a small risk register that scores six assets. Impact and likelihood are scored from 1 to 5 and multiplied, which is the simplest way to sort a long list of worries.

Python

```
ASSETS = [
    ("customer database", 5, 4),
    ("internal wiki",      4, 3),
    ("build server",       4, 4),
    ("marketing website",  3, 2),
    ("employee laptops",   4, 3),
    ("backup tapes",       5, 2),
]

def band(score):
    if score >= 15:
        return "high"
    if score >= 8:
        return "medium"
    return "low"

print("%-22s %9s %7s %6s" % ("asset", "impact", "likel.", "risk"))
print("-" * 48)
for name, impact, likelihood in ASSETS:
    score = impact * likelihood
    print("%-22s %9d %7d %6d  (%s)" % (name, impact, likelihood, score, band(score)))

top = max(ASSETS, key=lambda a: a[1] * a[2])
print()
print("highest scoring asset:", top[0], "->", top[1] * top[2])
low = min(ASSETS, key=lambda a: a[1] * a[2])
print("lowest scoring asset:", low[0], "->", low[1] * low[2])
```

Expected output (executed with Python 3.13):

output

```
asset                    impact  likel.   risk
------------------------------------------------
customer database             5       4     20  (high)
internal wiki                 4       3     12  (medium)
build server                  4       4     16  (high)
marketing website             3       2      6  (low)
employee laptops              4       3     12  (medium)
backup tapes                  5       2      10  (medium)

highest scoring asset: customer database -> 20
lowest scoring asset: marketing website -> 6
```

The scores are more honest than they look: they force a conversation about why the build server scores as high as it does, and that conversation is the real product of the exercise.

Example 2: the same register drawn as a risk matrix, which is how most teams present it to management.

output

```
              likelihood
          1     2     3     4     5
        +-----+-----+-----+-----+-----+
    5   |  5  | 10  | 15  | 20  | 25  |   customer db (5x4=20)
i       +-----+-----+-----+-----+-----+
m   4   |  4  |  8  | 12  | 16  | 20  |   build server (4x4=16)
p       +-----+-----+-----+-----+-----+
a   3   |  3  |  6  |  9  | 12  | 15  |   wiki, laptops (3x4=12)
c       +-----+-----+-----+-----+-----+
t   2   |  2  |  4  |  6  |  8  | 10  |   backups (2x5=10)
        +-----+-----+-----+-----+-----+
    1   |  1  |  2  |  3  |  4  |  5  |
        +-----+-----+-----+-----+-----+
```

Cells from 15 up are treated first; cells below 8 are documented and reviewed on a schedule.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Buying tools before listing assets	you protect what the tool covers, not what matters	start with an asset register
Treating every problem as critical	real critical work drowns in noise	score impact and likelihood, then sort
Confusing vulnerability with risk	an unpatched test box is not the same as an exposed database	separate likelihood and impact before deciding
Accepting risk silently	nobody can review a decision that was never written down	record who accepted it, why, and when it is revisited
Believing one control is enough	every control eventually fails for some user or some reason	layer controls and test each one
Reporting a problem with no owner	a vulnerability without a name next to it never gets fixed	name an owner and a due date for every finding

1.5 Practice exercises

1. List the three most valuable assets in a small online shop and rate the impact of losing each one from 1 to 5.
2. For each asset write one realistic threat and one weakness that would let the threat succeed.
3. Classify these controls: a door lock, a camera, an off-site backup, a security awareness poster, a firewall rule.
4. Using the scores in Example 1, decide which two assets you would work on this quarter, and justify the choice in two sentences.
5. Write an acceptance note for the marketing website risk: owner, reason and review date.

Solutions

1. Example: the customer order database (impact 5, it is the business), the payment data store (impact 5, regulated), the web server (impact 4, sales stop if it fails). Any ranking with a reason is acceptable; the reasoning matters more than the number.
2. Example: order database, threat of an external attacker exploiting a public web flaw, weakness of an out-of-date framework version. Or a careless employee, weakness of a shared login with no logging.
3. Door lock: preventive. Camera: detective. Off-site backup: corrective, and it also supports availability. Poster: deterrent. Firewall rule: preventive.
4. The customer database (score 20) and the build server (score 16) come first, because a compromise of either spreads to everything else the shop runs. The website scores lowest and can wait for a scheduled review.
5. Example: owner is the marketing manager; reason is that the site is static content restored from a repository within an hour; review date is six months from today.

1.6 Mini test

1. Which part of the CIA triad does an off-site backup support most directly? - a) Confidentiality - b) Integrity - c) Availability - d) None of them
2. True or false: a vulnerability always causes a breach.
3. What is the difference between a threat and a vulnerability?
4. Which control type notices an event that has already happened? - a) Preventive - b) Detective - c) Corrective - d) Deterrent
5. A company buys cyber insurance for risk it cannot reduce. Which treatment is that? - a) Avoid - b) Reduce - c) Transfer - d) Accept
6. Why does defence in depth matter if the outer layer is strong? - a) it does not, a strong layer is enough - b) every control can fail, so layers limit the damage - c) it only matters for large companies - d) it is required by the CIA triad

Answer key and explanations

1. c - backups restore service after loss or destruction, which is availability.
2. False - a vulnerability needs a threat and a path to reach it before anything happens.
3. A threat is the actor or event that wants to cause harm; a vulnerability is the weakness it uses to succeed.
4. b - detective controls such as alerts and logs reveal events after the fact.
5. c - insurance moves the financial consequence to another party, which is the definition of transferring risk.
6. b - layers assume failure elsewhere, so a single mistake is not fatal.

Chapter 2 - Cryptography basics: hashing, encoding and randomness

2.1 Summary

- A hash function turns any input into a fixed-size fingerprint that cannot be run backwards.
- The same input always produces the same digest, and changing one character produces a completely different one.
- Hashing answers two questions only: has this data changed, and is this the same input as before.
- Base64 changes the shape of data so it can travel through text systems; it uses no key, so it is encoding and not encryption.
- Passwords must be hashed with a slow algorithm such as Argon2, bcrypt or scrypt, never with a fast general-purpose digest.
- A unique random salt per password breaks precomputed tables and hides the fact that two people chose the same password.
- Random values used for security must come from the operating system entropy pool, not from a standard random number generator.
- When a service says passwords are stored safely, it means slow hashing with a per-user salt and a tunable cost.

2.2 Concepts

What a hash function is

A hash function takes input of any size and produces a digest of fixed size. Digests are useful because they are short, always the same for the same input, and impractical to reverse.

Property	What it means	Why defenders care
Deterministic	the same input always gives the same digest	you can re-check a file later
Fixed size	any input gives the same length	digests fit in a database column
One way	the input cannot be recovered from the digest	safe to store password verifiers
Avalanche	a tiny input change changes about half the bits	tampering is obvious
Collision resistant	finding two inputs with one digest is infeasible	a digest can stand in for a file

Which algorithm for which job

Algorithm	Verdict	Use it for
MD5	broken	nothing security related; old non-security checksums only
SHA-1	broken for security	nothing security related

Algorithm	Verdict	Use it for
SHA-256, SHA-512	current	file integrity, signatures, certificates
bcrypt	slow, adjustable	password storage
scrypt	slow, memory-hard	password storage
Argon2id	slow, memory-hard, modern	password storage

Fast digests are a feature for integrity checking and a bug for passwords: hardware can test billions of SHA-256 guesses per second.

Hashing, encoding and encryption compared

Job	Key?	Reversible?	Example
Hashing	no	no	SHA-256 of a download
Encoding	no	yes	base64 for email attachments
Encryption	yes	yes	AES with a secret key

Confusing them is the most common beginner mistake. Base64 hides nothing; the absence of a key means there is nothing to keep secret.

Salts, peppers and stretching

A salt is random data stored next to the digest and mixed with the password before hashing. A pepper is a secret value added to every password and kept in application configuration rather than the database. Stretching means choosing a work factor slow enough to make guessing expensive, typically around a tenth of a second per attempt, and raising it as hardware improves.

output

```
stored record for one user
+-------------------------------------------------+
| algorithm: argon2id                             |
| work factor: 3 iterations, 64 MiB memory        |
| salt: 16 random bytes, stored in clear          |
| digest: the result of hashing(salt + password)  |
+-------------------------------------------------+
```

The salt is not secret. Its job is to make every digest unique and to make precomputed tables useless.

Randomness

Security depends on unpredictable values: keys, salts, tokens, nonces. A general-purpose generator such as the `random` module is deterministic if the seed is known, so it must never be used for these. The `secrets` module and `os.urandom` both draw from the operating system.

2.3 Examples

Example 1: hashing in practice. The script shows the avalanche effect, the harm of a missing salt, the fix of a per-user salt, and the cost of stretching.

Python

```python
import hashlib

def sha256(text):
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

msg = "correct horse battery staple"
print("message :", msg)
print("sha256  :", sha256(msg))
print("length  :", len(sha256(msg)), "hex characters")

# A one-character change changes the whole digest (avalanche effect).
msg2 = "correct horse battery staplE"
print()
print("message :", msg2)
print("sha256  :", sha256(msg2))

# Hashing the same input always gives the same digest.
print()
print("repeat  :", sha256(msg))
print("same?   :", sha256(msg) == sha256(msg))

# Two users, same password, no salt: identical digests.
alice_pw = "Summer2026!"
bob_pw = "Summer2026!"
print()
print("alice   :", hashlib.sha256(alice_pw.encode()).hexdigest()[:32], "...")
print("bob     :", hashlib.sha256(bob_pw.encode()).hexdigest()[:32], "...")
print("equal digests?", hashlib.sha256(alice_pw.encode()).hexdigest()
      == hashlib.sha256(bob_pw.encode()).hexdigest())

# With a unique random salt per user the digests differ.
import os
salt_a = os.urandom(16)
salt_b = os.urandom(16)
digest_a = hashlib.sha256(salt_a + alice_pw.encode()).hexdigest()
digest_b = hashlib.sha256(salt_b + bob_pw.encode()).hexdigest()
print()
print("salt A  :", salt_a.hex())
print("salt B  :", salt_b.hex())
print("salted A:", digest_a[:32], "...")
print("salted B:", digest_b[:32], "...")
print("equal digests?", digest_a == digest_b)

# Key stretching: slow hash functions are deliberately expensive.
import time
start = time.perf_counter()
hashlib.pbkdf2_hmac("sha256", b"Summer2026!", b"salt-value", 200_000)
elapsed = time.perf_counter() - start
print()
print("pbkdf2_hmac 200000 iterations: %.3f s" % elapsed)
```

Expected output (executed with Python 3.13; the salts differ on every run and the timing depends on the machine):

output

```
message : correct horse battery staple
sha256  : c4bbcb1fbec99d65bf59d85c8cb62ee2db963f0fe106f483d9afa73bd4e39a8a
length  : 64 hex characters

message : correct horse battery staplE
sha256  : 4b0dd3ad2f2a318999c13ce2a63fe00f2963fe898eaf84bbb472e42b722c2026

repeat  : c4bbcb1fbec99d65bf59d85c8cb62ee2db963f0fe106f483d9afa73bd4e39a8a
same?   : True

alice   : a5c95e887dbf1c0a49fd01bced847d64 ...
bob     : a5c95e887dbf1c0a49fd01bced847d64 ...
equal digests? True

salt A  : 406645c58c1dbdf41a747c2654a33806
salt B  : 0575c24645b4ef56d53495226274ea64
salted A: 084fd6a3753e2281d4b9a8a8a0bb2a4e ...
salted B: 5c6e6c3bc7be9e17f6b61df70c24cf9a ...
equal digests? False

pbkdf2_hmac 200000 iterations: 0.319 s
```

Example 2: three jobs that beginners mix up, side by side.

output

```
hashing     "hello"  --(no key)-->  2cf24dba5fb0a30e26e83b2ac5b9e29e...
                                    (one way, fixed size)

encoding    "hello"  --(no key)-->  aGVsbG8=
                                    (reversible by anyone)

encryption  "hello"  --(key K)-->  f3 9a 01 c7 ...
                                    (reversible only with key K)
```

The fingerprint looks like the ciphertext, which is why the confusion happens, but a key changes everything: without a key there is no secrecy.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Storing passwords as SHA-256 digests	attackers test billions of guesses per second	use Argon2id, bcrypt or scrypt with a salt
One shared salt for the whole database	precomputed tables work again	generate a unique random salt per user
MD5 or SHA-1 for anything security related	collisions are practical to produce	SHA-256 and above, or a slow password hash
Calling base64 encryption	anyone can decode it in one command	real encryption with a managed key
Seeding a random generator for tokens	the sequence becomes predictable	use `secrets` or `os.urandom`
Hashing data that needs to be read back	digests cannot be reversed	encrypt instead, and back up the key safely

Mistake	Why it hurts	Best practice
Never raising the work factor	guessing gets cheaper every year	plan to increase cost on a schedule

2.5 Practice exercises

1. Two employees choose the password `Sunshine1`. Describe what an attacker sees if the database uses unsalted SHA-256, and what they see if it uses a unique salt per user.
2. How many hexadecimal characters long is a SHA-256 digest, and why does a fixed length help when comparing digests?
3. Give one sentence each for hashing and for encryption, and name one job for each in a web application.
4. A developer generates password reset tokens with the `random` module. Explain the problem and give the correct Python call.
5. Why is it acceptable to store the salt in the same database row as the digest?

Solutions

1. With unsalted SHA-256 both rows hold the same digest, so the attacker knows the passwords match, cracks the digest once and owns both accounts. With a unique salt per user the digests differ, each must be attacked separately, and the duplicate password stays hidden.
2. A SHA-256 digest is 64 hexadecimal characters, because it is 256 bits and each hex character carries 4 bits. Fixed length means every digest field has the same size, so comparisons and storage stay simple and predictable.
3. Hashing maps input to a fixed-size fingerprint that cannot be reversed, and it is used to verify file integrity. Encryption scrambles data with a key so that only key holders can read it, and it is used to protect backups.
4. The `random` module is deterministic: knowing or guessing the seed reveals the sequence, including tokens. Use `secrets.token_urlsafe(32)` or `os.urandom(32)`.
5. The salt is not meant to be secret. Its job is to make every digest unique so that precomputed tables and shared digests are useless. Security comes from the salt being random and unique, not from keeping it hidden.

2.6 Mini test

1. What does a hash function produce for a given input? - a) a random number - b) a fixed-size fingerprint that cannot be reversed - c) an encrypted copy of the input - d) a compressed version of the input
2. True or false: base64 encoding keeps data secret.
3. Which property means a one-character change alters about half the digest bits? - a) determinism - b) the avalanche effect - c) collision resistance - d) key stretching

4. Why are fast digests such as SHA-256 a poor choice for password storage? - a) they are reversible - b) they allow billions of guesses per second on normal hardware - c) they cannot use a salt - d) they produce digests that are too long
5. What is the job of a salt? - a) to encrypt the password - b) to make each digest unique and defeat precomputed tables - c) to speed up hashing - d) to replace the password
6. Which Python call is appropriate for a security token? - a) `random.randint(0, 999)` - b) `random.random()` - c) `secrets.token_hex(16)` - d) `math.floor(time.time())`

Answer key and explanations

1. b - a digest is a fixed-size one-way fingerprint of the input.
2. False - base64 is encoding; without a key there is no secrecy.
3. b - the avalanche effect is exactly that property.
4. b - speed is the attacker advantage; password hashes are meant to be slow and adjustable.
5. b - a unique salt makes identical passwords produce different digests and makes precomputed tables useless.
6. c - `secrets` draws from the operating system entropy pool, while the `random` module is predictable.